

CONSTRUCTING THE VISIBILITY GRAPH FOR n -LINE SEGMENTS IN $O(n^2)$ TIME

Emo WELZL *

Institute for Information Processing (IIG), Technical University of Graz and Austrian Computer Society, Schiesstattgasse 4a, A-8010 Graz, Austria

Communicated by L. Boasson

Received January 1985

Revised April 1985

Given a set S of line segments in the plane, its visibility graph G_S is the undirected graph which has the endpoints of the line segments in S as nodes and in which two nodes (points) are adjacent whenever they ‘see’ each other (the line segments in S are regarded as nontransparent obstacles). It is shown that G_S can be constructed in $O(n^2)$ time and space for a set S of n nonintersecting line segments. As an immediate implication, the shortest path between two points in the plane avoiding a set of n nonintersecting line segments can be computed in $O(n^2)$ time and space

Keywords: Computational geometry, visibility graph, shortest path

1. Introduction

In recent papers, Edelsbrunner, O’Rourke and Seidel [3] and Chazelle, Guibas and Lee [1] introduced an $O(n^2)$ time and space algorithm to compute the planar (subdivision) graph $G(A)$ induced by an arrangement A of n lines in the Euclidean plane. This algorithm has a number of applications, some of which rely on the following fact: Given a line h in an arrangement A , the (at most) $n - 1$ intersections of h with the other lines in A can be obtained sorted along h in $O(n)$ time from $G(A)$. That is, given A , these n sorted lists (one list for each of the n lines) can be obtained in $O(n^2)$ time, a bound which improves on the direct approach of applying n times an $O(n \log n)$ time sorting algorithm which amounts to $O(n^2 \log n)$ on the whole. Hence, for example, given n lines in the plane, one can decide in $O(n^2)$ time whether any three of them go through a common point.

However, in many cases, one requires all (at most) $\binom{n}{2}$ intersections of n lines sorted according

to their x -coordinates, rather than n ‘separate’ sorted lists. By the well-known dualism between arrangements of lines on the one side and configurations of points on the other side, this corresponds to sorting the slopes of the lines passing through any two of n points in the plane. For this problem(s) the best asymptotic upper time bound is still the one obtained by directly sorting the intersections (slopes) which takes $O(n^2 \log n)$ time.

In this informal paper we show that the partial order induced by $G(A)$ on *all* intersections (slopes) can be sufficient for some problems to obtain the $O(n^2)$ time bound. In particular, we show that for n nonintersecting line segments S in the plane, the *endpoint visibility graph* G_S of S can be constructed in $O(n^2)$ time and space.

This undirected graph G_S is defined on the set V_S of endpoints of the line segments in S as set of nodes. Two nodes (points) v and v' are adjacent in G_S if the open line segment with endpoints v and v' intersects none of the line segments in S (see Fig. 1 for an example).

Visibility graphs have been used several times as an approach to shortest path problems in the plane (see, e.g., Lozano-Perez and Wesley [7]): as soon as the visibility graph for n line segments is given, the shortest path problem between two

* This research was performed while the author was on leave at the Institute of Applied Mathematics and Computer Science, University of Leiden, 2300 RA Leiden, The Netherlands.

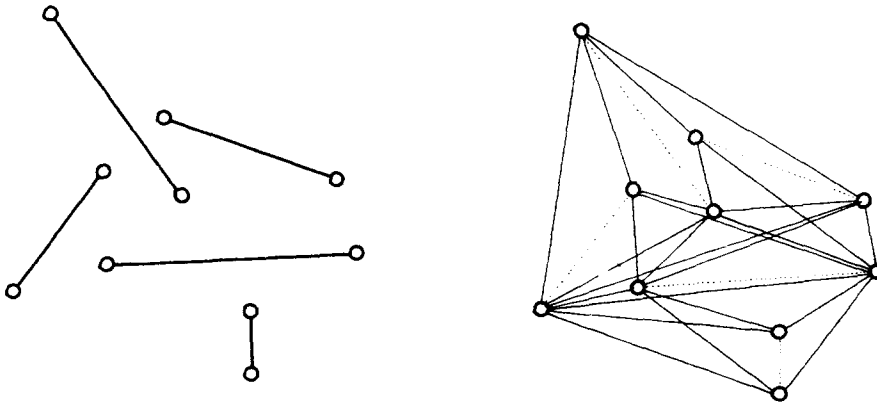


Fig. 1.

points can be solved in $O(n^2)$ time by using the standard single source shortest path algorithm for graphs (see Dijkstra [2]). The best known solutions for the construction of the visibility graph were $O(n^2 \log n)$ time algorithms (see, e.g., Lee [5] or Sharir and Schorr [9]). Thus this construction appeared to be the bottleneck of the shortest path algorithms in the plane and we get as an immediate application of our algorithm that we can compute a shortest path avoiding n nonintersecting line segments in the plane in $O(n^2)$ time (and space). Better solutions to the shortest path problem are known for special cases, as, e.g., an $O(n \log n)$ solution for parallel line segments is presented in [6].

2. Construction of visibility graph

Throughout this section let S denote a set of n nonintersecting closed line segments in the plane, let V_S be the set of endpoints of the line segments in S , and let G_S be the endpoint visibility graph of S with V_S as set of nodes and with the set of edges E_S (as defined in Section 1). We assume that the points in V_S are in general position, that is, no three points in V_S are collinear, no lines through points in V_S are parallel, and no two points in V_S lie on a common vertical line. This assumption is made mainly to keep this exposition simple—the presented algorithm can be generalized (without loss in the asymptotic time and space complexity bounds) to sets of line segments (i) for which there is no restriction on V_S , and (ii) in which line

segments possibly intersect in their endpoints.

Moreover, we use the following notation. For $v \in V_S$, let $s(v)$ be the segment in S of which v is an endpoint. For two points v, v' , let $d(v, v')$ be the direction determined by the directed line passing from v to v' . D_S denotes the sequence of the $\binom{2n}{2}$ sorted (by slope from $-\infty$ to $+\infty$) directions leading into the first and fourth quadrant defined by the points in V_S . (That is, for every pair of distinct nodes v and v' in V_S , either $d(v, v')$ or $d(v', v)$ is represented in D_S .)

We introduce two distinguished elements s_∞ and v_∞ which represent a line segment and a point, respectively, in infinity. S_∞ denotes $S \cup \{s_\infty\}$. By definition, every ray intersects s_∞ in point v_∞ .

The distance between two points v and v' is denoted by vv' ; if $v' = v_\infty$, then $vv' = \infty$. Finally, if $R \subseteq V_S \times S_\infty$ is a functional relation, then, for $v \in V_S$, vR denotes the unique element s in S_∞ with $(v, s) \in R$.

Let us consider the *initial view relation* $R_0 \subseteq V_S \times S_\infty$ which is defined as follows. For every point $v \in V_S$, let $(v, s) \in R_0$, where s is the first line segment intersected by the ray emanating from v in vertical direction downwards. (If no line segment in S is intersected, then (v, s_∞) is in R_0 !) Fig. 2 gives a graphical display of this relation.

Suppose now that we rotate the direction under consideration counterclockwise. Then the relation does not change until we 'sweep' through the first direction d in D_S . Even then the relation changes only locally. For $d = d(v, v')$, $v, v' \in V_S$, we have to distinguish between the following four cases (see also Fig. 3):

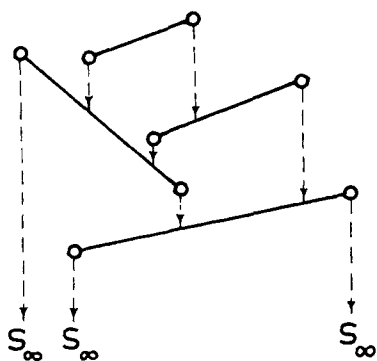


Fig. 2.

(i) If v and v' are endpoints of the same line segment, then the relation does not change.

(ii) If, seen from v in direction d , v' is in front of vR_0 , then v is now seeing $s(v')$, i.e., vR_0 is replaced by $s(v')$.

(iii) If v' is endpoint of vR_0 , then v is now seeing $v'R_0$.

(iv) If, seen from v in direction d , v' is behind vR , then the relation remains unchanged.

Note, moreover, that cases (ii) and (iii) imply that $\{v, v'\} \in E_S$ while cases (i) and (iv) imply that $\{v, v'\} \notin E_S$. This is the idea underlying the following definition.

Let $d = d(v, v')$, $v, v' \in V_S$, be a direction in D_S and let $R \subseteq V_S \times S_\infty$ be a functional relation. Then $rot(R, d)$ and $edge(R, d)$ are defined if the ray emanating from v through v' intersects vR in a point w , and that as follows (if $vR = s_\infty$, then $w = v_\infty!$):

if $s(v') = s(v)$ **then**

$edge(R, d) = \emptyset$ and $rot(R, d) = R$;

else if $\overline{vv'} < \overline{vw}$ **then**

$edge(R, d) = \{\{v, v'\}\}$ and

$rot(R, d) = R - \{(v, vR)\} \cup \{(v, s(v'))\}$;

else if $s(v') = vR$ (i.e., $w = v'$) **then**

$edge(R, d) = \{\{v, v'\}\}$ and

$rot(R, d) = R - \{(v, vR)\} \cup \{(v, v'R)\}$;

else if $\overline{vv'} > \overline{vw}$ **then**

$edge(R, d) = \emptyset$ and $rot(R, d) = R$.

Thus, if, for an arbitrary direction d_0 , we have a view relation R (as defined for vertical downwards above), and if we rotate this direction sweeping through a direction d (in D_S), then $rot(R, d)$ gives the correct new relation and $edge(R, d) = \{\{v, v'\}\}$ if and only if the points v and v' defining d see each other in S , i.e., $\{v, v'\} \in E_S$. So if we start with $R = R_0$, go through all directions in D_S , and update R to $rot(R, d)$ for every direction d , then the union of all corresponding $edge(R, d)$ gives exactly E_S . This should be enough explanation to show that the following algorithm correctly computes E_S , where $(d_1, d_2, \dots, d_m) = D_S$, $m = \binom{2n}{2}$.

Algorithm VGRAPH:

compute R_0 as defined above;

$E_S \leftarrow \emptyset$; $R \leftarrow R_0$;

for $i = 1$ **until** $\binom{2n}{2}$ **do**

$E_S \leftarrow E_S \cup edge(R, d_i)$;

$R \leftarrow rot(R, d_i)$;

enddo;

Let R be organized in a linear array, where—for an arbitrary enumeration $\{v_1, v_2, \dots, v_{2n}\}$ of V_S —the i th element represents v_iR , $1 \leq i \leq 2n$.

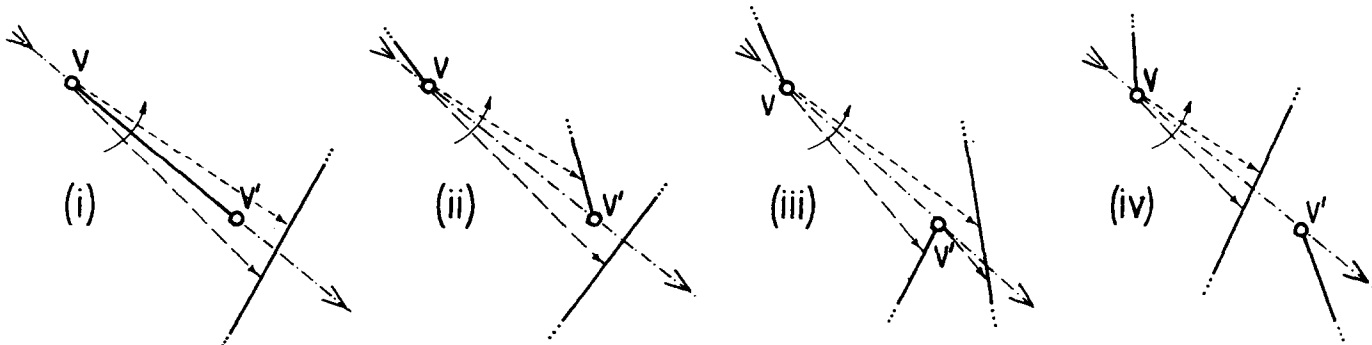


Fig. 3.

Then it is clear from the definition of *rot* and *edge* that every step in the loop of the algorithm can be computed in $O(1)$ time, which gives $O(n^2)$ time for the whole loop. R_0 can be computed in $O(n \log n)$ time by a standard sweep technique. Thus we are left with the computation of D_S as bottleneck, since the best known upper bound for this problem is $O(n^2 \log n)$. This can be settled as follows.

Two directions $d = d(v, w)$ and $d' = d(v', w')$ in D_S , $v, w, v', w' \in V_S$, are termed *independent* if $\{v, w\} \cap \{v', w'\} = \emptyset$.

What we mean by 'independent' becomes clear by the following lemma.

Lemma 1. *Let $R \subseteq V_S \times S_\infty$ be a functional relation and let d_1 and d_2 be two independent directions in D_S . Then $\text{rot}(\text{rot}(R, d_1), d_2)$ is defined if and only if $\text{rot}(\text{rot}(R, d_2), d_1)$ is defined, and if they are defined, then*

- (i) $\text{rot}(\text{rot}(R, d_1), d_2) = \text{rot}(\text{rot}(R, d_2), d_1)$,
and
- (ii) $\text{edge}(R, d_1) \cup \text{edge}(\text{rot}(R, d_1), d_2) =$
 $\text{edge}(R, d_2) \cup \text{edge}(\text{rot}(R, d_2), d_1)$.

Proof. Note that, for $d = d(v_1, v'_1)$, whether or not $\text{rot}(R, d_1)$ is defined depends on v_1 and $v_1 R$ only and, if it is defined, then $\text{rot}(R, d_1)$ differs from R only for v_1 (i.e., $wR = w \text{rot}(R, d_1)$ for all $w \in V_S - \{v_1\}$) and $v_1 \text{rot}(R, d_1)$ depends on $v_1 R$ and $v'_1 R$ only. So, if, for $d_2 = d(v_2, v'_2)$, $\{v_1, v'_1\} \cap \{v_2, v'_2\} = \emptyset$, then it makes no difference whether we modify R according to d_1 and then according to d_2 or vice versa. Hence the assertions of the lemma easily follow. $\square$

Lemma 1 shows that if we interchange two consecutive independent directions in D_S , then Algorithm VGRAPH still computes E_S correctly, and, of course, we can repeat such a manipulation still maintaining the validity of the sequence for Algorithm VGRAPH. This leads to the following definition.

A permutation D of D_S is called *good* if it can be obtained from D_S by repeatedly interchanging consecutive independent directions.

Lemma 2. (i) *Algorithm VGRAPH correctly computes E_S if it is started with a good permutation $(d_1, d_2, \dots, d_m)$ of D_S .* (ii) *A good permutation of*

D_S *can be computed in $O(n^2)$ time and space from S .*

Proof. (i) is clear from Lemma 1. To show (ii), consider the arrangement A_S of lines which is dual to the configuration of points in V_S , i.e., every point $v = (a, b)$ of V_S is mapped to a line T_v with equation $y = ax + b$. Then it is well known that every intersection (a', b') of two lines T_v and $T_{v'}$ in A_S corresponds to the line $y = -a'x + b'$ passing through v and v' in V_S . Hence, every direction in D_S with slope k corresponds to an intersection in A_S with x -coordinate $-k$ (and vice versa).

Let now $G(A_S)$ be the acyclic digraph with the intersections in A_S as nodes and with an arc from an intersection d to an intersection d' if and only if d is to the right of d' , and d and d' are adjacent on a line in A_S ('adjacent' means that d and d' lie on a common line and the open line segment with endpoints d and d' intersects only this line in A_S). Evidently, if there is no directed path between two intersections in $G(A_S)$, then their corresponding directions in D_S are independent. Also, if there is an arc from d to d' , then d precedes d' in D_S . Hence, a topological sort of the nodes in $G(A_S)$ corresponds to a good permutation of D_S .

$G(A_S)$ can be computed from V_S in $O(n^2)$ time and space (see, e.g., Edelsbrunner et al. [3] and Chazelle et al. [1]) and a topological sort of $G(A_S)$ can be computed in $O(n^2)$ time and space (see, e.g., Mehlhorn [8]). This yields the second statement of the lemma. $\square$

Now our result immediately follows.

Theorem. *Given a set S of n nonintersecting line segments (in general position) in the plane, the endpoint visibility graph G_S can be computed in $O(n^2)$ time and space.*

As pointed out in Section 1 we get the following direct consequence.

Corollary. *Given a set S of n nonintersecting line segments (in general position) and two points x and y in the plane, then the shortest path between x and y which is disjoint from S can be computed in $O(n^2)$ time and space.*

As already mentioned in the beginning of this section, the restriction that V_S must be in general position can be easily abandoned. Moreover, we could permit the line segments to intersect in their endpoints, thus obtaining an $O(n^2)$ algorithm for a shortest path between two given points avoiding a set of polygons with n bounding edges.

3. Discussion

We have demonstrated how the endpoint visibility graph for n nonintersecting line segments can be computed in $O(n^2)$ time by a 'walking around the set' algorithm, as it was used already in [4]. However, we have been able to show that in our case (i) the updates necessary when passing through a critical direction can be done in $O(1)$ time, and (ii) it is possible to go through the $\Theta(n^2)$ critical directions in an order which can be obtained in $O(n^2)$ time.

Since the endpoint-visibility graph may have $\Omega(n^2)$ edges for n line segments, it is tempting to claim that the presented algorithm is optimal. This is true for the worst case, if one requires a standard representation of the endpoint visibility graph (adjacency matrix or adjacency lists). However, there are a number of improvements possible.

(1) 'Dense' configurations of line segments may give rise to 'sparse' visibility graphs. The presented algorithm always needs $\Omega(n^2)$ time, independent of the size of the computed graph. Thus, there is still a need for an algorithm which is efficient for sparse visibility graphs.

(2) Possibly, endpoint visibility graphs have a succinct representation which can be computed in $o(n^2)$ time and from which necessary information can be quickly obtained. This is, of course, only

possible if there are less than $2^{o(n^2)}$ different visibility graphs on $2n$ nodes. So natural questions are: "Which graphs are endpoint visibility graphs?" and "How many of them exist?"

(3) Find an $O(n^2)$ time algorithm for properly intersecting line segments.

Another open problem which naturally (re-) arises in connection with this presentation: "Is it possible to compute the sorted list of the directions defined by n points in the plane in $O(n^2)$ time?"

References

- [1] B. Chazelle, L.J. Guibas and D.T. Lee, The power of geometric duality, in: Proc. 24th Ann. IEEE Symp. on Foundations of Computer Science (1983) 217–225.
- [2] E.W. Dijkstra, A note on two problems in connection with graphs, *Numer. Math.* 1 (1959) 269–271.
- [3] H. Edelsbrunner, J. O'Rourke and R. Seidel, Constructing arrangements of lines and hyperplanes with applications, in: Proc. 24th Ann. IEEE Symp. on Foundations of Computer Science (1983) 83–91.
- [4] H. Edelsbrunner, M. Overmars and D. Wood, Graphics in flatland: A case study, in: F. Preparata, ed., *Advances in Computing Research* 1 (1983) 35–59.
- [5] D.T. Lee, Proximity and reachability in the plane, Ph.D. Thesis, University of Illinois at Urbana-Champaign, 1978.
- [6] D.T. Lee and F.P. Preparata, Euclidean shortest paths in the presence of rectilinear barriers, *Networks* 14 (1984) 393–410.
- [7] T. Lozano-Perez and M.A. Wesley, An algorithm for planning collision-free paths among polyhedral obstacles, *Comm. ACM* 22 (1979) 560–570.
- [8] K. Mehlhorn, *Data Structures and Algorithms 2: Graph Algorithms and NP-Completeness* (Springer, Berlin-Heidelberg, 1984).
- [9] M. Sharir and A. Schorr, On the shortest path in polyhedral spaces, in: Proc. 16th. Ann. Symp. on Theory of Computation (1984) 144–153.